

Setting Up a Kubernetes Cluster with Kubeadm on AWS EC2

Complete Guide with Production Best Practices

Table of Contents

1. Prerequisites & Planning
2. AWS EC2 Setup & Security
3. Worker Node Configuration
4. Control Plane Setup
5. Cluster Initialization & Networking
6. Application Deployment
7. Production Best Practices
8. Monitoring & Logging
9. Troubleshooting Guide
10. Security Hardening

1. Prerequisites & Planning

System Requirements

Each node must meet the following minimum specifications:

2 GB RAM or more per machine (4GB+ recommended for production)

2 CPUs or more (4+ cores recommended for control plane)

Full network connectivity between all cluster nodes

Unique hostname, MAC address, and product_uuid for every node

20 GB minimum disk space (50 GB+ recommended)

Internet connectivity for downloading packages and images

Cluster Architecture Planning

This guide creates a single control-plane cluster with one worker node. For production, consider:

High Availability: Multiple control-plane nodes (3, 5, or 7 recommended)

Multiple worker nodes for workload distribution

Separate etcd cluster for critical deployments

Load balancer for control-plane endpoint

2. AWS EC2 Setup & Security Configuration

EC2 Instance Launch

Launch two Ubuntu 22.04 LTS instances (one for control-plane, one for worker) with the following configuration:

Instance Type: t3.medium (minimum) or t3.large (recommended)

AMI: Ubuntu Server 22.04 LTS

Root Volume: 50 GB (gp3)

VPC: Custom VPC with private subnets (recommended)

Availability Zone: Same for control-plane and worker (or multiple for HA)

IAM Roles for EC2 Instances

Create an IAM role with the following permissions:

```
{ "Version": "2012-10-17", "Statement": [ { "Effect": "Allow", "Action": [ "ec2:DescribeInstances", "ec2:DescribeVolumes", "ec2:DescribeTags", "ec2:DescribeSecurityGroups" ], "Resource": "*" }, { "Effect": "Allow", "Action": [ "elasticloadbalancing:DescribeLoadBalancers", "elasticloadbalancing:DescribeListeners" ], "Resource": "*" } ] }
```

Security Group Configuration

Control Plane Security Group Ingress Rules:

Protocol	Port(s)	Source	Description
TCP	6443	Worker nodes, kubectl clients	Kubernetes API
TCP	2379-2380	etcd peers	etcd
TCP	10250	kubelet, control plane	Kubelet API
TCP	10251	Scheduler	Scheduler
TCP	10252	Controller Manager	Controller Manager

Worker Node Security Group Ingress Rules:

Protocol	Port(s)	Source	Description
TCP	10250	Control plane	Kubelet API
TCP	30000-32767	External clients	NodePort Services
UDP	53	All nodes	DNS

NOTE: Allow all traffic between nodes within the cluster security groups.

3. Worker Node Configuration

Step 1: Set Hostname

```
hostnamectl set-hostname "k8s-node1.github.io" echo ""hostname -I | awk '{ print $1}'` `hostname`" >> /etc/hosts
```

Step 2: Update and Install Required Packages

```
sudo apt-get update sudo apt-get install git curl unzip tree wget -y
```

Step 3: Disable Swap (Critical for Kubernetes)

```
sudo swapoff -a sudo sed -i 's/^(\.*)$/#1/g' /etc/fstab
```

Step 4: Load Required Kernel Modules

```
sudo tee /etc/modules-load.d/containerd.conf <<EOF overlay br_netfilter EOF # Load kernel modules sudo modprobe overlay sudo modprobe br_netfilter
```

Step 5: Configure Kernel Parameters

```
sudo tee /etc/sysctl.d/kubernetes.conf <<EOF net.bridge.bridge-nf-call-ip6tables = 1 net.bridge.bridge-nf-call-iptables = 1 net.ipv4.ip_forward = 1 EOF # Reload the changes sudo sysctl --system
```

Step 6: Install containerd Container Runtime

```
# Install required dependencies sudo apt-get install -y curl gnupg2 software-properties-common apt-transport-https ca-certificates # Add Docker GPG key sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/trusted.gpg.d/docker.gpg # Add Docker repository sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" # Install containerd sudo apt-get update sudo apt-get install -y containerd.io
```

Step 7: Configure containerd for Kubernetes

```
# Generate default containerd configuration sudo containerd config default | sudo tee /etc/containerd/config.toml >/dev/null 2>&1 # Enable systemd cgroup (required for kubelet) sudo sed -i 's/SystemdCgroup \= false/SystemdCgroup \= true/g' /etc/containerd/config.toml # Restart containerd sudo systemctl restart containerd # Verify status sudo systemctl status containerd
```

Step 8: Add Kubernetes Repository

```
# Add Kubernetes GPG key curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.31/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg # Add Kubernetes repository echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.31/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list # Update package list sudo apt-get update
```

Step 9: Install Kubernetes Components

```
sudo apt-get install -y kubelet kubeadm kubectl
```

Step 10: Hold Kubernetes Packages to Prevent Auto-Upgrade

```
sudo apt-mark hold kubelet kubeadm kubectl
```

Step 11: Verify Component Versions

```
kubelet --version kubeadm version kubectl version --client
```

Step 12: Enable kubelet Service

```
sudo systemctl enable kubelet
```

4. Control Plane Node Setup

Steps 1-11 are identical to the Worker Node configuration. After completing those steps, proceed with:

Step 12: Pre-flight Checks

```
# Check required ports are available sudo netstat -tlnp | grep -E ':(6443|2379|2380|10250|10251|10252)' # Verify
swap is disabled sudo swapon --show # Should output nothing # Check kernel modules are loaded lsmod | grep -E
'overlay|br_netfilter' # Check sysctl settings cat /etc/sysctl.d/kubernetes.conf
```

Step 13: Initialize Kubernetes Cluster

```
# Initialize the cluster sudo kubeadm init \ --control-plane-endpoint=k8s-cp.github.io \ --pod-network-
cidr=192.168.0.0/16 \ --service-cidr=10.96.0.0/12 \ --apiserver-advertise-address=<CONTROL_PLANE_IP> >
~/k8s-cluster.output # Save the output - it contains the join command for worker nodes cat ~/k8s-cluster.output
IMPORTANT: The output will contain a join command. Save this - you'll need it to add worker nodes.
```

Step 14: Set Up kubeconfig for the Current User

```
mkdir -p $HOME/.kube sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config sudo chown $(id -u):$(id -g)
$HOME/.kube/config # Verify kubectl get nodes
```

Step 15: Install Calico CNI for Pod Networking

```
# Install Calico network plugin kubectl apply -f
https://raw.githubusercontent.com/projectcalico/calico/v3.25.0/manifests/calico.yaml # Wait for Calico pods to be
ready (2-3 minutes) kubectl wait --for=condition=ready pod -l k8s-app=calico-node -n kube-system --timeout=300s
```

Alternative: Flannel CNI (Lightweight Option)

```
kubectl apply -f https://raw.githubusercontent.com/coreos/flannel/master/Documentation/kube-flannel.yml
```

Step 16: Verify Cluster Status

```
# Check cluster info kubectl cluster-info # Check node status (should show control-plane as Ready) kubectl get
nodes # Check system pod status kubectl get pods --all-namespaces
```

5. Joining Worker Node to Cluster

Update /etc/hosts on Both Nodes

Control Plane node (/etc/hosts):

```
<CONTROL_PLANE_IP> k8s-cp.github.io <WORKER_IP> k8s-node1.github.io
```

Worker node (/etc/hosts):

```
<CONTROL_PLANE_IP> k8s-cp.github.io <WORKER_IP> k8s-node1.github.io
```

Join Worker Node to Cluster

Run on the Worker node (from kubeadm init output):

```
sudo kubeadm join k8s-cp.github.io:6443 \ --token <TOKEN> \ --discovery-token-ca-cert-hash sha256:<HASH>
If you forgot the join command, generate it on the control-plane:
```

```
kubeadm token create --print-join-command
```

Verify Worker Node Joined Successfully

Run on control-plane:

```
# Check node status kubectl get nodes # Check detailed node info kubectl describe node k8s-node1.github.io #  
Check if node is Ready kubectl get nodes -o wide
```

6. Deploying Applications on Kubernetes

Quick Deployment: Nginx

```
# Create deployment with 2 replicas kubectl create deployment nginx-app --image=nginx --replicas=2 # Expose as  
NodePort service kubectl expose deployment nginx-app --type=NodePort --port=80 # Check service and get  
NodePort kubectl get svc nginx-app # Access the application # http://<WORKER_IP>:<NodePort> # or #  
http://<CONTROL_PLANE_IP>:<NodePort>
```

Production Deployment: Using YAML Manifests

Create tomcat-deployment.yaml:

```
apiVersion: apps/v1 kind: Deployment metadata: name: tomcat-deployment namespace: default labels: app:  
tomcat spec: replicas: 2 selector: matchLabels: app: tomcat template: metadata: labels: app:  
tomcat spec: containers: - name: tomcat image: nandeesh10/tomcat:1.0.0 ports: -  
containerPort: 80 resources: requests: memory: "256Mi" cpu: "250m" limits:  
memory: "512Mi" cpu: "500m" livenessProbe: httpGet: path: / port: 80  
initialDelaySeconds: 30 periodSeconds: 10 readinessProbe: httpGet: path: / port: 80  
initialDelaySeconds: 5 periodSeconds: 5  
Create tomcat-service.yaml:
```

```
apiVersion: v1 kind: Service metadata: name: tomcat namespace: default labels: app: tomcat spec: type:  
LoadBalancer externalTrafficPolicy: Local ports: - name: http port: 80 protocol: TCP targetPort: 80  
selector: app: tomcat  
Deploy the Application:
```

```
# Apply deployment kubectl apply -f tomcat-deployment.yaml # Apply service kubectl apply -f tomcat-  
service.yaml # Check deployment status kubectl get deployment tomcat-deployment # Check pods kubectl get pods  
-l app=tomcat # Check service kubectl get svc tomcat # Scale the deployment kubectl scale deployment tomcat-  
deployment --replicas=3 # Update deployment image kubectl set image deployment/tomcat-deployment  
tomcat=nandeesh10/tomcat:2.0.0 # Rollback to previous version kubectl rollout undo deployment/tomcat-  
deployment
```

7. Production Best Practices

Resource Management

Always define resource requests and limits:

```
resources: requests: memory: "256Mi" cpu: "250m" limits: memory: "512Mi" cpu: "500m"
```

Pod Disruption Budgets for High Availability:

```
apiVersion: policy/v1 kind: PodDisruptionBudget metadata: name: tomcat-pdb spec: minAvailable: 1 selector:  
matchLabels: app: tomcat
```

Node Auto-Scaling Strategy

For AWS, use Karpenter or Cluster Autoscaler:

Monitor node CPU/Memory utilization

Automatically scale up when pods are pending

Scale down idle nodes to reduce costs

Namespace Isolation

```
# Create namespaces for different environments kubectl create namespace production kubectl create namespace staging kubectl create namespace development # Deploy to specific namespace kubectl apply -f deployment.yaml -n production
```

Network Policies

```
apiVersion: networking.k8s.io/v1 kind: NetworkPolicy metadata: name: deny-all spec: podSelector: {} policyTypes: - Ingress --- apiVersion: networking.k8s.io/v1 kind: NetworkPolicy metadata: name: allow-tomcat spec: podSelector: matchLabels: app: tomcat policyTypes: - Ingress ingress: - from: - namespaceSelector: matchLabels: name: production ports: - protocol: TCP port: 80
```

RBAC (Role-Based Access Control)

```
apiVersion: v1 kind: ServiceAccount metadata: name: app-deployer namespace: production --- apiVersion: rbac.authorization.k8s.io/v1 kind: Role metadata: name: app-deployer namespace: production rules: - apiGroups: ["apps"] resources: ["deployments", "statefulsets"] verbs: ["get", "list", "watch", "create", "update", "patch"] --- apiVersion: rbac.authorization.k8s.io/v1 kind: RoleBinding metadata: name: app-deployer namespace: production roleRef: apiGroup: rbac.authorization.k8s.io kind: Role name: app-deployer subjects: - kind: ServiceAccount name: app-deployer namespace: production
```

8. Monitoring & Logging

Install Prometheus for Metrics Collection

```
# Add Prometheus Helm repository helm repo add prometheus-community https://prometheus-community.github.io/helm-charts helm repo update # Install Prometheus stack helm install prometheus prometheus-community/kube-prometheus-stack \ -n monitoring --create-namespace # Access Prometheus kubectl port-forward -n monitoring svc/prometheus-kube-prometheus-prometheus 9090:9090
```

Install ELK Stack for Centralized Logging

```
# Install Elasticsearch helm install elasticsearch elastic/elasticsearch -n logging --create-namespace # Install Kibana helm install kibana elastic/kibana -n logging # Install Filebeat (collects logs from pods) helm install filebeat elastic/filebeat -n logging
```

Enable Kubernetes Audit Logging

```
# Create audit policy file sudo tee /etc/kubernetes/audit-policy.yaml <<EOF apiVersion: audit.k8s.io/v1 kind: Policy rules: - level: RequestResponse omitStages: - RequestReceived resources: - group: "" resources: ["pods", "services", "deployments"] - level: Metadata omitStages: - RequestReceived EOF # Update kube-apiserver configuration to use audit logs # Edit: /etc/kubernetes/manifests/kube-apiserver.yaml
```

Key Metrics to Monitor

Node metrics:

CPU and memory usage

Disk I/O and network bandwidth

Pod metrics:

Container CPU and memory

Pod restart count

Cluster metrics:

API server latency

etcd performance

Failed pod count

9. Troubleshooting Guide

Common Issues & Solutions

Issue	Possible Cause	Solution
Nodes stuck in NotReady state	Container runtime not running or kubelet service down	<code>sudo systemctl status containerd</code> && <code>sudo systemctl restart kubelet</code>
Pods pending indefinitely	Resource constraints or node selector mismatch	<code>kubectl describe pod <pod-name></code> && <code>kubectl top nodes</code>
Network connectivity issues	CNI plugin not installed or misconfigured	<code>kubectl get pods -n kube-system</code> && <code>kubectl logs <cni-pod></code>
Control plane components not running	Insufficient system resources or corrupted manifests	<code>sudo systemctl status kubelet</code> && <code>sudo journalctl -u kubelet -n 50</code>
High API server latency	etcd performance issues or resource constraints	<code>kubectl top nodes</code> && <code>kubectl get events -n kube-system</code>

Debugging Commands Cheat Sheet

```
# Check cluster status kubectl cluster-info kubectl get nodes kubectl get nodes -o wide # Inspect specific resources
kubectl describe node <node-name> kubectl describe pod <pod-name> -n <namespace> kubectl describe
deployment <deployment-name> # View logs kubectl logs <pod-name> -n <namespace> kubectl logs <pod-name>
-n <namespace> --previous # Previous container logs kubectl logs -f <pod-name> # Follow logs in real-time #
Access pod shell kubectl exec -it <pod-name> -n <namespace> -- /bin/bash # Check resource usage kubectl top
nodes kubectl top pods -n <namespace> # View events kubectl get events -n <namespace> kubectl get events -n
<namespace> --sort-by='.lastTimestamp' # Troubleshoot network connectivity kubectl run -it --rm debug --
image=busybox -- sh # Inside container: nslookup kubernetes.default
```

System-Level Debugging

```
# Check kubelet logs sudo journalctl -u kubelet -n 100 -f # Check containerd logs sudo journalctl -u containerd -n
100 -f # Check kube-apiserver logs sudo tail -f /var/log/pods/kube-system_kube-apiserver-*/kube-apiserver/0.log #
Check system resources free -h df -h top -u kubernetes # Network diagnostics sudo netstat -tlnp | grep -E ':(6443|
10250|10251|10252)' sudo iptables -L -n
```

10. Security Hardening

API Server Security

Enable encryption at rest for etcd:

```
# Generate encryption key head -c 32 /dev/urandom | base64 # Create encryption config file sudo tee
/etc/kubernetes/enc.yaml <<EOF apiVersion: apiserver.config.k8s.io/v1 kind: EncryptionConfiguration resources: -
resources: - secrets providers: - aescbc: keys: - name: key1 secret: <YOUR_GENERATED_KEY>
- identity: {} EOF
```

Enable TLS encryption for all components:

Pod Security Policies

```
apiVersion: policy/v1beta1 kind: PodSecurityPolicy metadata: name: restricted spec: privileged: false
allowPrivilegeEscalation: false requiredDropCapabilities: - ALL volumes: - 'configMap' - 'emptyDir' -
'projected' - 'secret' - 'downwardAPI' - 'persistentVolumeClaim' hostNetwork: false hostIPC: false hostPID:
false runAsUser: rule: 'MustRunAsNonRoot' seLinux: rule: 'MustRunAs' supplementalGroups: rule:
'MustRunAs' fsGroup: rule: 'MustRunAs' readOnlyRootFilesystem: false
```

Image Security Best Practices

Use private container registries with authentication

Scan images for vulnerabilities before deployment

Use image pull secrets:

```
# Create docker registry secret kubectl create secret docker-registry regcred \ --docker-server=<registry-url> \ --
docker-username=<username> \ --docker-password=<password> \ --docker-email=<email> # Reference in
deployment apiVersion: apps/v1 kind: Deployment metadata: name: private-app spec: template: spec:
imagePullSecrets: - name: regcred containers: - name: app image: private-registry/app:v1.0
```

Secrets Management

```
# Never store secrets in version control # Use Kubernetes secrets kubectl create secret generic db-credentials \ --
from-literal=username=dbuser \ --from-literal=password=secretpassword # Or use external secrets with HashiCorp
Vault # helm install external-secrets external-secrets/external-secrets -n external-secrets-system --create-namespace
```

Network Security

Implement strict network policies (shown in Section 7)

Use service mesh for advanced traffic management (Istio, Linkerd)

Enable mTLS between pods

Audit & Compliance

Enable API audit logging (shown in Section 8)

Regularly review RBAC permissions

Monitor for suspicious activities

Keep Kubernetes components updated

Appendix: Additional Resources

Useful Commands Reference

```
# View all available resources kubectl api-resources # Get detailed API documentation kubectl explain deployment
# Export configuration kubectl get deployment -o yaml > backup.yaml # Dry-run to validate manifests kubectl
apply -f deployment.yaml --dry-run=client # Get shell in existing pod kubectl debug <pod-name> -it # Check DNS
resolution inside cluster kubectl run -it --rm debug --image=busybox -- nslookup kubernetes.default # Scale
workloads kubectl scale deployment <name> --replicas=5 # Set resource limits kubectl set resources deployment
<name> --limits=cpu=500m,memory=256Mi # Perform rolling update kubectl set image deployment/<name>
<container>=<image:tag> # View deployment history kubectl rollout history deployment/<name> # Rollback to
previous version kubectl rollout undo deployment/<name>
```

Recommended Helm Charts for Common Requirements

Ingress Controller: ingress-nginx

Monitoring: kube-prometheus-stack

Logging: elasticsearch, kibana, filebeat

Package Management: Helm

Container Registry: Harbor, Nexus

Secrets Management: Vault, External Secrets Operator

Service Mesh: Istio, Linkerd

Useful Online Resources

Official Kubernetes Documentation: <https://kubernetes.io/docs/>

Kubeadm Setup Guide: <https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/>

Kubernetes Best Practices: <https://kubernetes.io/docs/concepts/configuration/overview/>

Security Best Practices: <https://kubernetes.io/docs/concepts/security/>

Cost Optimization Tips

Use Reserved Instances (RIs) for long-term costs savings

Implement resource quotas to prevent over-provisioning

Use Spot Instances for non-critical workloads

Enable cluster autoscaling to right-size nodes

Monitor and clean up unused persistent volumes

Use node affinity to pack workloads efficiently